

TP 2 d'Interprétation et compilation

Christian Rinderknecht

7-8 octobre 2003

Ce TP prolonge le précédent.

1. Reconsidérez la sémantique opérationnelle de la division (sans spécification des erreurs). L'ordre d'évaluation des opérandes n'y est pas spécifié. Réécrivez la règle pour qu'elle force l'évaluation de son second opérande avant le premier. Pour cela, inspirez-vous de la règle de la liaison locale.

L'implantation fidèle de cette nouvelle sémantique est maladroite. Proposez une variation simple du code pour la division qui répond au problème.

2. Ajoutez les fonctions et les applications. Notez bien que le type des valeurs doit maintenant changer. Introduisez les modifications nécessaires. N'oubliez pas d'enrichir le traitement d'erreurs dans les cas où on attend un entier et on obtient en fait une fonction, ainsi que le cas contraire. Testez. (*Gardez une trace de vos cas de test.*) Par exemple :

```
1+2*(if true then let f = fun y -> y+1 in f(3+4) else 5)
```

3. Implantez les fonctions récursives natives. Testez par exemple

```
let rec f = fun n -> ifz n then 1 else n*f(n-1) in f 7  
let rec x = x in x
```

4. Soit e l'arbre de syntaxe abstraite correspondant au programme 1 2. Existe-t-il un environnement ρ et une valeur v tels que $\rho \vdash e \rightarrow v$? Quelle différence voyez-vous entre cet exemple et $(\text{fun } f \rightarrow f\ f)\ (\text{fun } f \rightarrow f\ f)$, donné dans le cours? Vérifiez votre raisonnement en soumettant ces expressions à votre calculette.
5. Vérifiez que les constructions **let** $x = e_1$ **in** e_2 et $(\text{fun } x \rightarrow e_2)\ e_1$ sont équivalentes du point de vue de l'évaluation — c'est-à-dire que l'une produit une valeur v si et seulement si l'autre produit également v .
6. Ajoutez au langage des liaisons globales de la forme **let** $x = e$.
7. Implantez la sémantique avec références.
8. Vérifiez que les constructions **let** $x = e_1$ **in** e_2 et $(\text{fun } x \rightarrow e_2)\ e_1$ sont équivalentes du point de vue du typage monomorphe, c'est-à-dire que l'une admet le type τ sous l'environnement Γ si et seulement si l'autre admet aussi τ sous Γ .

9. Peut-on typer les expressions ci-dessous de façon monomorphe et, si oui, avec quels types ?

```

let f = fun x → x in f f
fun f → f f
1 2
let f = fun x → x in let x = f 1 in f (fun x → x + 1)

```

10. Implantez l'axiome FUN-OPT qui restreint l'environnement dans les fermetures aux variables libres de la fonction :

$$\rho \vdash \text{Fun}(x, e) \text{ as } f \Rightarrow \text{Clos}(x, e, \rho|\mathcal{L}(f)) \quad \text{FUN-OPT}$$